Stage C preliminary results: the linear probe

9 November 2026, for Gabriel and Léonard. Supersedes every probe number quoted in chat before this document. Nine verification agents re-derived the findings from the raw per-cell data on 9 November; the figures and wordings below are the ones that survived.

1. What this is, and what it is not

Fourteen pretrained encoders exist. None can emit a BBBP or ESOL prediction, because each ends in a 1,613-wide Mordred-descriptor head. A **linear probe** is the cheapest way to get a downstream number out of them without the fine-tuning loop: freeze the encoder, take one 2,048-wide vector per molecule, and fit a linear model on the frozen scaffold splits.

This is not the protocol's fine-tuning path and the numbers are not interchangeable with it. A probe cannot adapt the encoder, so it tends to *understate* what pretraining bought. It is also not comparable to the figures in the lecture or in CheMeleon's paper, both of which fine-tune. Read it for the **shape** of the size effect, not the level.

Its purpose was to answer one question a week before the real curve could: *is there a slope at all, and where does it turn?* That matters because Stage D has to pick an operating size, and running Stage D at 25,000 rather than 100,000 molecules costs about a quarter as much.

2. Where everything is

Nothing below is in git: `.gitignore` excludes `runs/*`. The scripts that produce it are committed; the outputs live on the cluster.

what	path
Size curve figure	runs/probe/figures/probe_size_curve.png
Label curve figure	runs/probe/figures/probe_label_curves.png - invalid, see D2
Aggregated results, 88 cells	runs/probe/results_summary.csv
Per-run results, 720 rows	runs/probe/results_per_run.csv
Per-cell predictions	runs/probe/<arm>__<task>__split<n>__n<labels>/predictions.csv
Per-cell metadata	same directory, manifest.json
Full verification report	/tmp/claude-270366/-home-gdupuis/59c491c8-1432-409f-9782-3bd74631459c/tasks/w9vg6widh.output
Probe implementation	scripts/probe.py
Figure code	scripts/plot_probe.py
Job script	slurm/probe.sbatch

3. How the numbers were produced

Pretraining, already complete. CheMeleon's recipe unchanged - a Chemprop D-MPNN, width 2048, depth 6, 8.71 M encoder parameters - trained to regress 1,613 Mordred descriptors with 15 percent of targets scored per step. Fourteen conditions: nested PubChem subsets at 500, 1k, 5k, 10k, 25k, 50k and 100k molecules, two seeds each. All fourteen share one frozen objective of 1,562 usable targets standardised against the 100,000-molecule condition, so the target does not widen as the corpus grows. **66.6 GPU-hours, 14 of 14 finished.**

The probe. For each of 18 arms and both tasks, every molecule is pushed through the frozen encoder and the 2,048-wide vector kept. This reproduces `MPNN.fingerprint` exactly - message passing, then

aggregation; the reference model is built with `batch_norm=False` so its `bn` is an identity - which is what makes the arms comparable to each other.

Then for every (arm, task, split seed 0-4, label count 50/100/500/all): fit logistic regression (BBBP) or ridge (ESOL), choose the regularisation strength on the **validation** split from a ten-point grid, never on test, and score on test. **720 cells, about 14,400 fits, 5 minutes on one L40S.**

The arms.

arm	what it controls for
14 pubchem conditions	the variable under study
released	CheMeleon's published 944,286-molecule encoder - top of the curve
random	same architecture, untrained - separates pretraining from capacity
mordred	~1,600 computed descriptors - the baseline the protocol says matters most
morgan	2,048-bit fingerprints - a conventional non-neural baseline

Splits. Five frozen scaffold splits per task, generated once and reused by every arm. BBBP 1,631 / 203 / 205, ESOL 902 / 112 / 114. Verified on all ten: zero index overlap and **zero shared Murcko scaffolds** between train and test.

---

4. Results

BBBP, AUROC at full labels (higher is better)

arm	size	mean	sd	n
released	944,286	0.9373	0.0254	5
pubchem	25,000	0.9366	0.0283	10
pubchem	50,000	0.9366	0.0231	10
pubchem	100,000	0.9317	0.0273	10
pubchem	10,000	0.9307	0.0276	10
mordred	-	0.9257	0.0238	5
pubchem	5,000	0.9156	0.0287	10
morgan	-	0.9145	0.0218	5
random	-	0.9057	0.0378	5
pubchem	1,000	0.9036	0.0413	10
pubchem	500	0.8794	0.0506	10

ESOL, RMSE at full labels (lower is better)

arm	size	mean	sd	n
mordred	-	0.8874	0.1729	5
pubchem	10,000	0.8983	0.1006	10
pubchem	25,000	0.9064	0.1021	10
pubchem	5,000	0.9792	0.1991	10
pubchem	50,000	0.9877	0.1236	10
pubchem	100,000	1.1400	0.2933	10
pubchem	1,000	1.1926	0.2728	10
random	-	1.4108	0.5275	5
pubchem	500	1.4123	0.5411	10
released	944,286	1.5209	0.9079	5
morgan	-	1.6602	0.0723	5

The aggregation is faithful: all 720 cells were independently re-scored and the published summary reproduces to 1.3e-15 with no count mismatches. **The arithmetic is not in question. The interpretation was.**

---

5. What the numbers mean

The headline a careful reviewer would accept

On a frozen linear probe over five scaffold splits, BBBP AUROC improves with PubChem pretraining-corpus size up to roughly 10,000-25,000 molecules and then flattens. ***Nothing in this experiment distinguishes any arm above 10,000 molecules from any other****, or from the released 944,286-molecule checkpoint, or from Mordred descriptors. The one contrast that clears the experiment's own resolution is that the 500-molecule corpus is worse than the top of the curve.* ***ESOL yields no readable statement about pretraining scale at all.***

The resolution, which is the whole story

- Honest 95 percent resolution on BBBP, including the encoder-draw random

effect: **0.034 AUROC**. (0.017 if one wrongly treats the pretraining draw as fixed.)

- Of 55 BBBP pairwise orderings, 22 reach p < 0.05 unadjusted and

none survive Benjamini-Hochberg or Bonferroni.

- Restricted to the protocol's pre-specified family - each arm against the

from-scratch control - 5 of 10 reach p < 0.05 unadjusted and **0 of 10 survive correction**. That is a null, and d `ocs/protocol.md` says a clear null is reported as one.

- Only 8 of the 55 pairs exceed the resolution, and **all 8 involve the

500-molecule arm**. No adjacent step on the size curve resolves.

- Per-cell test-set sampling noise alone is 0.011-0.017 AUROC over 205 test

molecules, so about half the resolution is irreducible until more molecules are scored.

What does survive

The trend. +0.057 AUROC from 500 to 25,000 molecules, paired across the five splits, t(4) = 5.74, **p = 0.005**, consistent in all five. Per-split Spearman between AUROC and log size averages **+0.76**, p = 0.0001. The curve is real; its fine structure is not.

Saturation, stated carefully. No arm above 10,000 is distinguishable from any other or from released. But the paired 25k-vs-released difference is -0.0007 with a 95 percent CI of **[-0.022, +0.021]** - so they are *indisting uishable*, not equal, and the interval still admits a gain from 38x more data as large as the whole 5k->25k improvement. The knee cannot be placed between 10k and 25k specifically: that step is +0.006, p = 0.25.

Mordred returns a null in both directions. 25k - mordred is +0.0109, p = 0.151. The comparison the protocol calls the one that matters most does not show the pretrained encoder winning *or* losing.

What does not survive, and was wrongly claimed earlier

claim made in chat	verdict
"500 molecules actively hurts"	Fails. Pooled Welch p = 0.283. Rides on one of two pretrain seeds (s0 -0.012 p=0.40, s1 -0.041); the between-seed gap 0.029 exceeds the claimed effect 0.026. Does not replicate on ESOL (+0.002, p=0.99). Reverses at 50 labels.
"Mordred beats every pretrained condition on ESOL"	False. pubchem_n10000_s0 scores 0.866 against Mordred's 0.887. Invisible in the summary because evaluate.py averages the two pretrain seeds.
"Released is worse than untrained on ESOL"	Fails. Paired +0.11, 95% CI [-0.37, +0.59], released better on 3 of 5 splits. One molecule carries 91 percent of split 0's squared error (predicted -34.5 against true -3.0); drop it and that cell goes 3.09 -> 0.92. Under MAE released is better 5 of 5.

"Pretraining helps most at low labels"	Weaker form only, and the attribution must go. The interaction is 0.115, 95% CI [-0.010, +0.240], p = 0.063. Against the same control at 50 labels, Morgan gains +0.109 and Mordred +0.124 - so the gain is largely not specific to pretraining. The 25k arm beats Mordred by 0.022, p = 0.27.

Safe wording for the low-label result, if it is used at all: *a linear probe on the 25,000-molecule encoder beats the from-scratch control by 0.146 AUROC at 50 labels (95% CI 0.01 to 0.28) and by 0.031 at full labels (CI 0.005 to 0.056); the difference between those gaps is 0.115 (95% CI -0.01 to 0.24), positive in all five splits but not resolved at n = 5.*

6. Defects found, and which cells they affect

Invalidating

D1. On ESOL the encoder-draw noise exceeds the entire size effect. The sd of a single pretrain draw is **0.2375 RMSE** against an sd across the seven size means of **0.1863**. Pairing on split cannot remove it, because the pretrain seed is not shared across size arms. The 95 percent detectable paired difference on ESOL is **0.59 RMSE**; the largest adjacent step is 0.22 and the full range is 0.51. *Affects every ESOL size-curve statement at every label count.* On BBBP the same component is 0.0152 against 0.0214 - comparable, not dominant, which is why BBBP partly survives.

Distorting

D2. The low-label columns are not low-label measurements. `probe.py:342` subsets the training labels but line 362 passes the **full** validation split. A BBBP "n=50" cell fits on 50 labels and consumes **253** (5.1x nominal); ESOL n=50 consumes 162. No manifest records `n_val`. *Affects all 540 cells at label counts 50/100/500 and makes `probe_label_curves.png` unusable.* Arm-vs-arm differences at a fixed label count survive; the levels and the slope do not.

D3. One divergent fit controls a published cell. `esol, pubchem, 500, n=50` reports 3.6867 ± 5.8623. Nine of the ten runs are 1.40-2.34; the tenth is **20.35**, a ridge fit that diverged and predicts -188.94 against a label range of [-9.33, 1.10]. `evaluate.py` takes an unguarded mean. This is the only one of 88 cells where sd exceeds the mean.

D4/D5. ESOL RMSE is dominated by single extrapolations. 40 of 360 ESOL cells score worse than a constant predictor. Eleven cells have one point carrying more than half the squared error. Rankings reverse under MAE.

D6. `evaluate.py` drops `pretrain_seed`. 280 of 720 rows share a full key with another row, so the two encoder draws are indistinguishable after aggregation and per-condition winners cannot be seen. `count=10` also overstates replication: it is 5 shared splits x 2 draws.

D7. The from-scratch control is one network, not five. `encoder_random()` defaults to seed 0 and the features are cached, so all five "random" runs share one initialisation with **zero measured initialisation variance** - and it is the protocol's designated comparator, and the highest-variance arm on the board.

D8. Grid pinning persists at 17.8 percent on BBBP and is arm-asymmetric (morgan 40 percent). All pinned cells are at label counts 50/100/500, **none at all-labels**, so the headline is unaffected.

D9. Only Mordred has test-informed preprocessing. `clean()` computes column medians and the zero-variance mask over train+val+test before any split. No label is involved, so this is not label leakage - but the one arm the protocol says matters most is the only one given a transductive advantage.

Minor, but disclose

- **D10.** `docs/protocol.md:156` is wrong: BBBP's 99 acyclic molecules land in **validation for seed 0**, not train in all five. Seed 0 is therefore a structurally different experiment and contributes to the error bar.

- **D11.** Pretraining provenance is unrecorded: checkpoints come from four commits, epoch counts at matched size differ widely (n500: 224 vs 37), and `runs/pretrain/*/manifest.json` records no epoch, step or final loss.

- **D12.** `scripts/probe.py` has no tests, and its resume cache has no key for code version or grid, so a resume could silently mix grids.

Not defects, checked and clean

No split or feature misalignment - the 11 RDKit-unparseable BBBP rows never enter any split. No train/test or scaffold contamination on any of the ten splits. Metrics on the right quantities. Regularisation selected on validation and demonstrably worse than a test oracle. `embed()` matches chemprop's `fingerprint`. All 18 arms share identical frozen test sets.

Benchmark exposure, measured rather than argued

PubChem contains 73 percent of BBBP and 93 percent of ESOL by structure, and the exposed fraction grows with corpus size - 0.0 percent of BBBP test molecules at 500, 7-8 percent at 100,000 - so it covaries with the x-axis. Deleting every exposed test molecule and rescoring moves BBBP AUROC by at most **0.0011**, against a rise of 0.057 that it would have to explain.

size	as reported	exposure removed	delta
500	0.8794	0.8794	+0.0000
10,000	0.9307	0.9302	-0.0005
25,000	0.9366	0.9355	-0.0011
100,000	0.9302	0.9291	-0.0011

Exposure is real and does grow with size, but it cannot account for the finding. This tests exact structure matches only; near-analogues are what the `--exclude-benchmarks` arm and Stage E's similarity binning are for.

---

7. What to fix before any of this is reported

In priority order:

1. **D2** - subsample the validation split alongside train, or select by k-fold inside the n labels. Until then no low-label number is a low-label number.
2. **D7** - give the from-scratch control five initialisations, one per split seed. It is the protocol's comparator and currently carries no variance.
3. **D6** - have `evaluate.py` carry `arm` and `pretrain_seed` through, so paired differences against the baseline are reconstructible as the protocol requires.
4. **D9** - fit `clean()` on train only, inside the split.
5. **D3/D5** - report median absolute error alongside RMSE on ESOL, and flag any cell where one point carries more than half the squared error.

Each is small. The sweep itself is 5 minutes and the feature cache survives, so a corrected re-run is cheap.

---

8. What the probe cannot answer

It cannot give a number comparable to the lecture or to CheMeleon's paper, because those fine-tune. It cannot measure what a fine-tuned encoder would do, and it systematically understates it. It cannot settle

ESOL at all, because the encoder-draw noise there exceeds the effect.

What it *can* do, and did: tell us the curve's shape a week early, and that the shape flattens by 10,000-25,000 molecules. If that holds under fine-tuning, **Stage D runs at 25,000 rather than 100,000**, which is roughly a quarter of the pretraining cost for its three arms. On that basis the probe has already paid for itself, whether or not any of its numbers appear in the write-up.

The real curve still needs `scripts/finetune.py`, which remains a 36-line stub. Four of the five stages run through it.